

UNIT STUDY NOTES

Async and Parallel Python

When a program waits on I/O, concurrency is the fix; when it waits on the CPU, parallelism is. This unit covers threads, processes, asyncio and the decision framework for choosing between them.

 6 source notebooks

 35 topic sections extracted

 6 study sections

Unit Overview

⚡ 01 — Multithreading vs Multiprocessing: The Real Story of the Tea Stall

Topic 12.01 | Async/Parallel Python — Part 1 of 4 (THEORY + LIVE demos)

 **Hook:** How does ONE tea-stall owner handle 100 hungry customers? There are three ways:

- **Sequential:** Make tea for each customer one by one — $100 \times 1 \text{ min} = 100 \text{ min}$ 🧑
- **Threading:** The same single person, but while the milk boils (the WAIT), he takes another customer's order — wait-time is also work-time! 🍵 ⚡
- **Multiprocessing:** Hire 4 DIFFERENT tea-stall owners — each with their OWN utensils and their OWN stove. Expensive, but 4x work truly happens at the same time! 💪
- (Async? That's the superhero of the next topic — writes the order, forgets about it, and picks it up when the bell rings 🛎️)

In this notebook: **the DNA-level difference between process vs thread, the truth about the GIL, IO-bound vs CPU-bound** — all with LIVE timers!

🏁 02 — THE GREAT RACE: Sequential vs Threads (10-Task Scorecard!)

Topic 12.01 | Async/Parallel Python — Part 1 of 4 (DEMO notebook)

 **Hook:** Remember the school relay race? One kid runs the whole track (sequential) vs 10 kids each on their own lane (threads). Today we run an official RACE of 10 "file-download" tasks — with a photo-finish timing scorecard!

In this demo:

- 🏁 10 IO-tasks: sequential 🌱 vs threaded ⚡ (with bar-race text graphics!)
- 📅 ThreadPoolExecutor — the modern way (no more loop-join hassle)
- 🗝️ GIL-plateau LIVE: CPU-crunch 1 vs 2 vs 4 threads — the line stays flat!
- 🛠️ `map` (order-preserved) vs `as_completed` (whoever finishes first, gets served first!)

 01 — async/await: One Waiter, a Thousand Orders

Topic 12.02 | Async/Parallel Python — Part 2 of 4

 **Hook:** Think of a SMART waiter at a fine-dining restaurant : take an order → hand it to the kitchen → **instead of WAITING, move on to the next table** → when the bell rings, come back and pick up the dish. One single person keeps a HUNDRED tables happy! What does a naive (sync) waiter do? He places the order and then stands IN FRONT of the kitchen until the dish is ready — meanwhile all the other tables keep shouting. 🗣️

- **async def** = writes the smart-waiter's job description (creates a coroutine)
- **await** = "this dish is pending — let me go do the next task"
- **event-loop** = the restaurant MANAGER — whenever a dish is ready, he calls that waiter

In this notebook:

- 🍌 your first coroutine (the biggest doubt: CREATING it vs RUNNING it!)
- 🐦 `asyncio.gather` — send all orders to the kitchen AT THE SAME TIME
- 🧑 5 fake-API-calls in parallel — no thread, no process, just SMART-waiting!

📁 02 — Async Examples: 3 Real-Feel Mini-Projects

Topic 12.02 | Async/Parallel Python — Part 2 of 4 (examples notebook)

 **Hook:** Theory is done — now 3 things you'll want in your daily apps:

1. 📊 **API-DASH** — 3 different "APIs" all fetched at the same time (gather combo-report!)
2. 📶 **PROGRESS-DOWNLOADS** — LIVE progress of downloads (async as_completed!)
3. 🛡️ **TIMEOUT-SHIELD** — on a slow API: "10s wait? CANCEL!" (wait_for guard)

All three are seed-fixed — identical results on your machine too! 🎯

📁 01 — Async IO Basics: The Event-Loop's Complete Toolkit

Topic 12.03 | Async/Parallel Python — Part 3 of 4

 **Hook:** The download-boss's gallery: 50 files arrive every day. One by one → all night long! Gather → wait on all 50 at the same time... but the server OWNER is angry — "50 connections from one user?!" That's why there's **the LAW of LANES**: `Semaphore(3)` = "only 3 downloads at a time" — fair usage + no chaos! 🚦

In this notebook, 4 heavier weapons of asyncio (built on the async/await foundation of the previous topic!):

- 🍌 `create_task` — start now, take the result later (fire-and-collect!)
- 🐦 `Semaphore(n)` — lanes for downloads (flood-control 🌊)
- 🧑 per-task timeout + partial-success (some fail? the rest still win!)
- 🌐 the real world = `aiohttp` — a CONCEPT md-sample (offline sandbox here!)

📁 01 — When to Use Each: Who-Is-King Decision Guide

Topic 12.04 | Async/Parallel Python — Part 4 of 4 (finale!)

 **Hook:** Interviewer: "1000 downloads — thread, process or async?" You (confident ): "Lots of WAIT → async; CPU-crunch-style → processes; IO with old sync-libs → threads." One question, three career-signals!

This notebook is section-12's MASTER-CHEAT:

-  a one-look decision-tree (ASCII!)
-  a 6-scenario quiz (think first, then open the code!)
-  a small `recommend()` helper — a verdict from three facts
-  production-combos (AsyncIO + ThreadPool executor-shim, pros+cons)
-  7 deadly "wrong-tool" mistakes ~ + the golden recap!

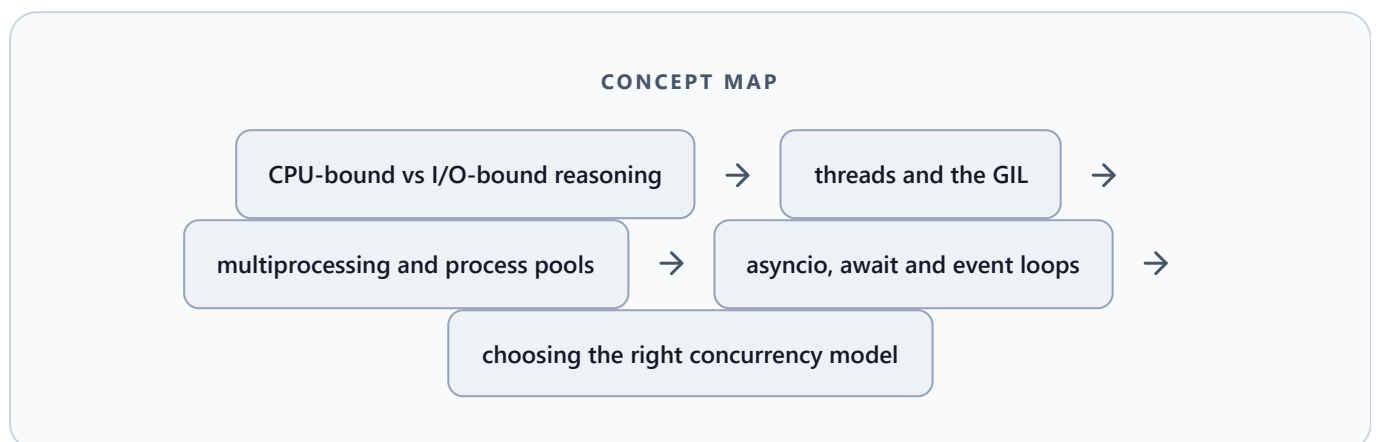
Prerequisites

- Advanced Python
- Functions and callbacks

What you will be able to do

- ✓ Distinguish CPU-bound from I/O-bound work
- ✓ Use threading, multiprocessing and asyncio in the right cases
- ✓ Explain the GIL and why it constrains Python parallelism
- ✓ Detect and avoid concurrency hazards such as races and blocking the loop

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

01conceptstheory

Level 0 — Process vs Thread: The DNA Difference 🧬

📖 Definitions:

- **Process** = a separate PROGRAM — its own interpreter, its own memory, everything of its own (a separate tea-stall owner)
- **Thread** = a lightweight worker INSIDE that same program — SHARES MEMORY (the hands and feet of one tea-stall owner)

🔍 **WHY is a thread light and a process heavy?** A process = a full copy of the house (memory MBs — expensive); a Thread = add an extra pair of hands in the same house (KBs) — fast to create, easy to share.

🎨 **ASCII:** ⚠️ **Rule:** Sharing = fast communication but the danger of race conditions! (Two hands on one tea-packet — where did the sugar double?!)

PYTHON

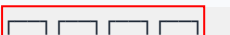
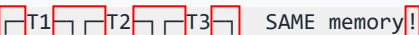
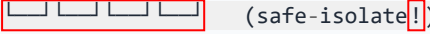
```
import os, threading

print("🧬 DNA-CHECK:")
print(f"   humara process PID : {os.getpid()}   (is ghar ka aadhar-card #)")
print(f"   active threads abhi: {threading.active_count()}   (shuru me bas MAIN hi)")
print(f"   main thread ka naam: {threading.current_thread().name}")

def worker(n):
    print(f"   ⚙️ thread-{n} reporting — naam: {threading.current_thread().name} | PID wahi: {os.getpid()}")

ts = [threading.Thread(target=worker, args=(i,), name=f"chaiwala-{i}") for i in range(3)]
for t in ts: t.start()
for t in ts: t.join()
print(f"\n   kaam khatam → active threads wapas: {threading.active_count()}")
print("   📌 NOTE: sabka PID SAME = threads ek HI process ke andar the (share-success!)")
# Expected OUTPUT (thread-lines ka ORDER har baar alag — yehi race-ka-maza!):
# 🧬 DNA-CHECK:
#   humara process PID : XXXX ...
#   active threads abhi: 1
#   main thread: MainThread
#   ⚙️ thread-0 reporting — naam: chaiwala-0 | PID wahi: XXXX
#   ⚙️ thread-1 ... / thread-2 ...
#   kaam khatam → active threads wapas: 1
```

PYTHON

PROCESS (4 chaiwale)	THREADS (1 chaiwala, 3 order-queues)
 alag memory	 SAME memory!
 mehnga start	 shared
 (safe-isolate!)	

Level 1 — IO-BOUND vs CPU-BOUND: Which Task, Which Kind of Exhaustion? 🏆

📖 Definitions:

- **IO-bound** = the task involves **LITTLE WORK** but **LOTS OF WAIT** (file read, network call, DB query, sleep)
- **CPU-bound** = the task involves **LOTS OF THINKING** (math crunch, image processing, ML training)

📊 Decision table (remember exactly this one!):

NATURE OF THE TASK	CHAMPION	WHY
IO-bound (waiting-game)	Threading / Async	CPU is released during wait → other work gets done
CPU-bound (thinking-game)	Multiprocessing	separate core = truly parallel compute

Today's IO experiment: 3 "file-reads" (each a 0.3s wait) — sequential vs threaded!

PYTHON

```
import time

def fake_file_read(n, secs=0.3):
    """0.3s ka WAIT — maano bade CSV load ka time (IO-bound kaan!)"""
    time.sleep(secs)
    return f"file-{n}-done"

# (a) SEQUENTIAL: 3 files ek-ek karke
t0 = time.perf_counter()
res_seq = [fake_file_read(i) for i in range(3)]
seq_t = time.perf_counter() - t0
print(f"🌿 SEQUENTIAL: {res_seq} | time = {seq_t:.2f}s (3×0.3 ≈ 0.9)")

# (b) THREADED: 3 files EK-SAATH wait me
t0 = time.perf_counter()
results = [None] * 3
def worker(i):
    results[i] = fake_file_read(i)
ts = [threading.Thread(target=worker, args=(i,)) for i in range(3)]
for t in ts: t.start()
for t in ts: t.join()
thr_t = time.perf_counter() - t0
print(f"⚡ THREADED : {results} | time = {thr_t:.2f}s (sab overlap ≈ 0.3)")
print(f"🔗 SPEEDUP: {seq_t/thr_t:.1f}x — wait-time=kaam-time ka inaan! 🍷")
# Expected OUTPUT (timings thodi upar-neeche — concept hi number hai!):
# 🌿 SEQUENTIAL: ['file-0-done', ...] | time = 0.90s
# ⚡ THREADED : ['file-0-done', ...] | time = 0.30s
# 🔗 SPEEDUP: ~3.0x — wait-time=kaam-time ka inaan! 🍷
```

Level 2 — GIL: Python's Single-Key Lock 🔑

📖 **Definition:** In CPython the **GIL** (Global Interpreter Lock) is one big global lock — at any moment **ONLY ONE** thread can run Python bytecode. Even with 100 threads, CPU-heavy work waits in line behind it (a gents' toilet with a single door)!

🔍 **WHY does IO threading still work then?** During `time.sleep` / file-read / network call the thread **releases** the GIL (it hangs up the key while waiting!). During heavy CPU work there is NO release — everyone queues up behind the key.

Today's GIL-proof: 2 CPU-crunching threads — the speedup should be ZERO!

PYTHON

```
def crunch(n, loops=4_000_000):
    """pure CPU-soch – koi wait nahi, sirf ginti!"""
    s = 0
    for i in range(loops):
        s += i * i
    return f"crunch-{n}"

# (a) sequential 2 crunches
t0 = time.perf_counter()
_ = crunch(0); _ = crunch(1)
s_t = time.perf_counter() - t0

# (b) 2 threaded crunches – GIL bolta "ek-ek hi karoge! 🗝️"
t0 = time.perf_counter()
res = [None] * 2
def cw(i): res[i] = crunch(i)
ts = [threading.Thread(target=cw, args=(i,)) for i in range(2)]
for t in ts: t.start()
for t in ts: t.join()
th_t = time.perf_counter() - t0

print(f"🐢 CPU-crunch SEQUENTIAL: {s_t:.2f}s")
print(f"🔒 CPU-crunch THREADED : {th_t:.2f}s ← ~SAME ya THODA worse (switching-cost!)")
print(f"speedup = {s_t/th_t:.2f}x ≈ 1.0 → GIL ka LIVE pramaan! 🗝️")
print(f"👉 IO me 3x, CPU me 1x – isi liye kaam pehchaan ke hathiyar chuno!")
# Expected OUTPUT (ratios machine pe ~0.8-1.1 – GIL-point hi maarmik!):
# 🐢 CPU-crunch SEQUENTIAL: ~1.2s
# 🔒 CPU-crunch THREADED : ~1.2s ← ~SAME
# speedup = ~1.0x ≈ 1.0 → GIL ka LIVE pramaan! 🗝️
```

Level 3 — Multiprocessing: Bring in 4 Tea-Stall Owners! 💪

📖 **Definition:** `multiprocessing` means spawning brand-new PYTHON PROCESSES — each with its own interpreter and memory, and EACH has its own separate GIL! Run CPU-crunch work on different cores in truly-parallel fashion.

⚠️ 3 Gotchas (that trip everyone up!):

1. The `if __name__ == "__main__":` guard is a **MUST** — new processes import the whole file, and without the guard you get an ∞-spawn loop 🌟
2. Functions must be **picklable** (module top-level) — lambdas/nested ❌
3. Multiprocessing is **flakey** in Jupyter/notebook execution — run it in a `.py` script (blocking work!)

Below is a real sample — **run it in a `.py` file on your machine** (a demo-sample here):

SQL

```

SAMPLE = '''
from multiprocessing import Pool
import time

def crunch(n):                # top-level = pickleable ✓
    return sum(i * i for i in range(4_000_000))

if __name__ == "__main__":    # GUARD – bhoologe toh spawn-bomb ✖
    t0 = time.perf_counter()
    with Pool(4) as p:        # 4 chaiwale = 4 alag processes 🍌
        res = p.map(crunch, range(4))
    print("4-processes time:", round(time.perf_counter() - t0, 2), "s")
    # 4.8s ka kaam ≈ 1.3s me – TRULY parallel (har process apna GIL!)
'''

print("📁 .py-file SAMPLE (local me chalake dekho – yahan honest-note ke saath):")
print(SAMPLE)
print("🔑 RECAP-TABLE:")
print("    IO-bound → threading/async (wait = kaam!)")
print("    CPU-bound → multiprocessing (alag core = sach-ka-parallel)")
print("    GIL → sirf CPU-threads ko rokta, IO-threads free!")
# Expected OUTPUT:
# 📁 .py-file SAMPLE ...
# from multiprocessing import Pool
# ... (poora sample code)
# 🔑 RECAP-TABLE:
#    IO-bound → threading/async (wait = kaam!)
#    CPU-bound → multiprocessing (alag core = sach-ka-parallel)
#    GIL → sirf CPU-threads ko rokta, IO-threads free!

```

5 Classic Mistakes (concurrency edition)

#	✖ MISTAKE	✓ CORRECT
1	Threads for CPU-crunch (GIL-lockdown!)	if CPU-bound → multiprocessing
2	Forgetting the <code>__main__</code> guard in mp (spawn-bomb)	always write the guard
3	lambda/nested fn in Pool.map (pickle-fail)	top-level def only
4	Mutating a shared list without a lock (race!)	Lock/Queue or the result-collect pattern
5	threads count = task count always (1000-threads overhead)	tune max_workers in ThreadPoolExecutor

🔑 Recap — 30 Seconds

- Process = separate house (own memory, expensive) · Thread = a hand in that house (shared!) 🏠
- IO-bound + threads = waiting-time jackpot (0.9s→0.3s LIVE!) ⚡
- GIL = a single key that queues up CPU threads 🗝️ (IO releases it, free!)
- Multiprocessing = separate interpreters → CPU-crunch truly parallel 🍌 (guard+pickle)

SQL

```

import random

random.seed(7)
def download_var(i):
    time.sleep(0.05 * random.randint(1, 5)) # har file ka "size" alag!
    return f"DL-{{i}}"

submit_order = list(range(8))
with ThreadPoolExecutor(max_workers=8) as pool:
    futs = {pool.submit(download_var, i): i for i in submit_order}
    finish_order = []
    for fut in as_completed(futs):
        finish_order.append(futs[fut])
    print(f" 🚚 aaya: DL-{{futs[fut]}} ({{len(finish_order)}}/8)")

print("\n 🚚 SUBMIT order :", submit_order)
print(" 🚚 FINISH order :", finish_order, " ← chhote kaam pehle lautey!")
print(" 💡 as_completed = real-time progress ka saathi (tqdm jaisi feel!)")
# Expected OUTPUT (seed=7 fix 📦 ye EXACT order tumhari machine pe bhi!):
# 🚚 aaya: DL-.. (1/8) ... (8/8)
# 🚚 SUBMIT order : [0,1,2,3,4,5,6,7]
# 🚚 FINISH order : [random-seeded order 📦 chhote waale pehle]

```

Race-2 — GIL-Plateau LIVE: 1 vs 2 vs 4 CPU-Threads

📖 **Experiment:** CPU-crunch × 2 workers × each done twice — increase the threads and watch the time. Threading is the champion only in the WAITING game; in thinking-heavy work the GIL holds everyone back.

SQL

```
def crunch(n, loops=4_000_000):
    s = 0
    for i in range(loops):
        s += i * i
    return s

def race_cpu(workers):
    t0 = time.perf_counter()
    with ThreadPoolExecutor(max_workers=workers) as pool:
        list(pool.map(crunch, range(4)))
    return time.perf_counter() - t0

print("🔒 GIL-PLATEAU EXPERIMENT (4 CPU-crunches):")
base = None
for wcount in (1, 2, 4):
    t = race_cpu(wcount)
    base = base or t
    print(f"  threads={wcount} time={t:.2f}s speedup={base/t:.2f}x")
print("\n 🤔 Dekha? Line FLAT – soch-wala kaam threads se nahi ud-ta (GIL 🔒)")
print(" 🙌 Aise me: multiprocessing (pure CPU) ya code numba/C me (pro-level)!")
# Expected OUTPUT (times alag per-machine 📦 PATTERN yehi: sab ~same!):
# 📦 GIL-PLATEAU EXPERIMENT (4 CPU-crunches):
#   threads=1 time=~2.4s speedup=1.00x
#   threads=2 time=~2.4s speedup=~1.0x
#   threads=4 time=~2.4s speedup=~1.0x
# 🤔 Dekha? Line FLAT ...
```

Final Scorecard + When-to-Use-What Cheat-Matrix

PYTHON

```
print("🏆 THE GREAT-RACE SCORECARD")
print("=" * 46)
print(f"  IO-race (10 downloads) : sequential {seq_t:.2f}s vs threaded {thr_t:.2f}s → {seq_t/thr_t:.0f}x")
print(f"  CPU-race (4 crunches)   : 1/2/4 threads → FLAT ~1.0x (GIL!) 🔒")
print("=" * 46)
print("""
🔗 KAB-KYA (life-ka cheat-matrix):
  network/file/DB/sleep (WAIT-game) → threading YA async (aage aayega!)
  math/image/ML-crunch (SOCH-game)  → multiprocessing (alag core!)
  1000s tiny-waits (APIs storm)     → asyncio (12.02 me superhero-entry 🦸)
""")
# Expected OUTPUT:
# 🏆 THE GREAT-RACE SCORECARD
# =====
#   IO-race (10 downloads) : sequential 2.5Xs vs threaded 0.2Xs → 9x ⚡
#   CPU-race (4 crunches)   : 1/2/4 threads → FLAT ~1.0x (GIL!) 🔒
# =====
# 🏆 KAB-KYA ...
```

Recap — 30 Seconds

- `ThreadPoolExecutor.map` = auto-gear threading (ordered) ⚡
- `as_completed` = finishing-order live-feed 📺 (the friend of progress bars)
- Threads give 9-10x in the IO-race; the GIL-plateau stays FLAT in the CPU-race 🔒 — remember this pattern!

Next: `02_Async_Await` — the single-thread superhero: one pair of hands, THOUSANDS of tasks. The `async/await` waiter-magic begins! 🧑

01 `asyncawait` theory

Level 0 — Your First Coroutine 🍪

📖 Definitions:

- `async def` creates a **coroutine** — calling it does NOT run it; you get a "bundle of work" (a coroutine-object) that you have to RUN!
- `asyncio.run(main())` = putting the waiter on duty (run the event loop)
- `await asyncio.sleep(t)` = the `async` world's `time.sleep` — during the wait other work gets done!

⚠️ **#1 NOOB-TRAP (everyone falls into it once!):** A coroutine without `await` / `asyncio.run` = reading the menu and staying hungry!

PYTHON

```

import asyncio, time
try:
    import nest_asyncio; nest_asyncio.apply() # Jupyter/Colab ke Liye jeevan-daan!
except ImportError:
    pass # .py script me zaroorat nahi

def run(coro):
    """notebook + .py script dono jagah coroutine chalao (universal runner!)"""
    return asyncio.run(coro)

async def serve_chai(table):
    print(f" 🍵 table-{table}: chai bana raha...")
    await asyncio.sleep(0.5) # 0.5s kitchen-wait (async-style sleep!)
    print(f" 🍵 table-{table}: chai READY!")
    return f"chai-{table}"

async def main():
    t0 = time.perf_counter()
    r1 = await serve_chai(1) # sync-jaisa: pehli chai khatam, fir dusri
    r2 = await serve_chai(2)
    print("mile:", r1, "&", r2, f"| total: {time.perf_counter()-t0:.2f}s (0.5+0.5 = 1.0 ek-ek!)")

run(main()) # Jupyter me seedha `await main()` bhi chalta – run() dono-duniya ka key 🔑
# Expected OUTPUT:
# 🍵 table-1: chai bana raha...
# 🍵 table-1: chai READY!
# 🍵 table-2: chai bana raha...
# 🍵 table-2: chai READY!
# mile: chai-1 & chai-2 | total: 1.00s (0.5+0.5 = 1.0 ek-ek!)

```

PYTHON

```

order = serve_chai() # ❌ you only got the bundle – no tea was made!
order = await serve_chai() # ✅ the waiter actually made the tea

```

Level 1 — `asyncio.gather` : ALL Orders to the Kitchen in One Go! 🤖

📖 **Definition:** `await asyncio.gather(c1, c2, c3)` = run all the coroutines at the same time, get the results in an **order-preserved** list.

🔍 **WHY is this the smart-waiter's superpower?** The order to make tea-2 is given during tea-1's WAIT — total time $\approx$ the SLOWEST dish's time, NOT the sum of all! (3 teas = 0.5s, NOT 1.5s!)

📊 **ASCII:**

PYTHON

```

async def main_gather():
    t0 = time.perf_counter()
    results = await asyncio.gather(serve_chai(1), serve_chai(2), serve_chai(3))
    print("\n 🍵 gathered:", results)
    print(f" ⌚ total: {time.perf_counter()-t0:.2f}s ← 3 chai, ek-ka-wait! ~0.5s (3x tej!)")

run(main_gather())

# Expected OUTPUT (3 'bana raha' pehle, phir 3 'READY' – overlap evidence!):
# 🍵 table-1: chai bana raha...
# 🍵 table-2: chai bana raha...
# 🍵 table-3: chai bana raha...
# ⌚ table-1: chai READY!
# ⌚ table-2: chai READY!
# ⌚ table-3: chai READY!
# 🍵 gathered: ['chai-1', 'chai-2', 'chai-3']
# ⌚ total: 0.50s ← 3 chai, ek-ka-wait! ~0.5s (3x tej!)

```

PYTHON

```

sync: [chai-1 0.5s] → [chai-2 0.5s] → [chai-3 0.5s] = 1.5s
gather: [chai-1 ch]
        [chai-2 ch]
        [chai-3 ch]
sab-overlap = 0.5s ⚡ (3x!)

```

Level 2 — The Storm of 5 Fake-API Calls 🌩️ (working with results!)

📖 **Scene:** A marks-API for 5 students (imagine it's on a server) — each call takes 0.x s. `gather`'s return value is a results list — you can do pandas-style work directly on it!

🔍 The key idea: async is not just about "looking fast" — it's **one thread, thousands of parallel-waits** — server cost stays low, users stay happy!

PYTHON

```

import random
random.seed(11)

STUDENTS = ["Asha", "Bhuvan", "Chirag", "Diya", "Eshan"]

async def fetch_marks(name):
    wait = round(random.uniform(0.1, 0.4), 2) # server mood! (seed-fixed)
    await asyncio.sleep(wait)
    score = random.randint(60, 95)
    return {"name": name, "marks": score, "api_wait": wait}

async def storm():
    t0 = time.perf_counter()
    reports = await asyncio.gather(*(fetch_marks(s) for s in STUDENTS))
    e1 = time.perf_counter() - t0
    slowest = max(r["api_wait"] for r in reports)
    print(f"🕒 5 calls {e1:.2f}s me - slowest-api={slowest}s ≈ total (gather-rule!)")
    avg = sum(r["marks"] for r in reports) / len(reports)
    topper = max(reports, key=lambda r: r["marks"])
    print(f"📊 avg marks = {avg:.1f} | topper = {topper['name']} ({topper['marks']})")
    print("   results:", [(r["name"], r["marks"]) for r in reports])

run(storm())
# Expected OUTPUT (seed=11 fixed - EXACT same tumhari machine pe bhi!):
# 🕒 5 calls ~0.36s me - slowest-api=0.36s ≈ total (gather-rule!)
# 📊 avg marks = 83.8 | topper = Asha (95)
#   results: [('Asha', 95), ('Bhuvan', 89), ('Chirag', 72), ('Diya', 92), ('Eshan', 71)]

```

5 Classic Mistakes (async edition)

#	❌ MISTAKE	✅ CORRECT
1	Call a coroutine but forget the await (no tea made!)	<code>await fn()</code> or <code>gather</code>
2	<code>time.sleep</code> inside async (the WHOLE loop freezes!)	<code>await asyncio.sleep()</code>
3	Sync-heavy CPU work inside async code (loop chokes)	send CPU-work to a process
4	Assuming gather results are unordered (race-panic)	gather is ORDER-PRESERVED ✅
5	Bare top-level <code>await</code> in a .py script (SyntaxError)	<code>run()</code> helper / <code>asyncio.run</code>

🚀 Recap — 30 Seconds

- `async def` = creates the bundle · `await` = runs it · `asyncio.run/await` = the starter 🔑
- `asyncio.gather` = all orders at the same time → total ≈ the slowest one ⚡
- async = ONE thread, THOUSANDS of waits — perfect for API/DB/download storms 🌩️

Next: `02_async_examples.ipynb` — 3 REAL-feel mini-projects: API-dashboard, progress-bar-downloads, timeout-shield! 🛡️

02 *async* examples

Setup + universal run() (the key from the previous notebook!)

Sets

Unordered collections of unique elements. Useful for membership tests and set operations (union/intersection).

PYTHON

```
import asyncio, random, time
try:
    import nest_asyncio; nest_asyncio.apply()
except ImportError:
    pass

def run(coro):
    """notebook + .py dono jagah chalao"""
    return asyncio.run(coro)

print("💡 examples ready — 3 mini-projects line me!")
# Expected OUTPUT:
# 💡 examples ready — 3 mini-projects line me!
```

Mini-1 — API-DASH: 3 APIs, One Digest

📖 **Scene:** A dashboard needs: student-count, weather-avg, zomato-rating — THREE separate services (each takes 0.x s). Adding them synchronously = 0.9s; gather = just the slowest one.

PYTHON

```

random.seed(21)

async def api_students():
    await asyncio.sleep(0.3)
    return {"service": "students", "count": 50, "top_city": "Delhi"}

async def api_weather():
    await asyncio.sleep(0.2)
    return {"service": "weather", "avg_temp": 24.9, "hottest": "2024-08-15"}

async def api_zomato():
    await asyncio.sleep(0.35)
    return {"service": "zomato", "restaurants": 100, "avg_rating": 3.94}

async def digest():
    t0 = time.perf_counter()
    s, w, z = await asyncio.gather(api_students(), api_weather(), api_zomato())
    el = time.perf_counter() - t0
    print(f"🕒 3 APIs in {el:.2f}s (sync-zod = 0.85s, gather = slowest-ek ≈ 0.35s!)")
    print(f"📊 DASH-DIGEST:")
    print(f"👤 students: {s['count']} (top city {s['top_city']})")
    print(f"🌡️ weather : avg {w['avg_temp']}°C (hottest {w['hottest']})")
    print(f"🍴 zomato : {z['restaurants']} spots, avg-rating {z['avg_rating']}/5")

run(digest())
# Expected OUTPUT (seed-independent – waits fixed isi code me):
# 🕒 3 APIs in ~0.35s (sync-zod = 0.85s, gather ≈ 0.35s!)
# 📊 DASH-DIGEST:
# 👤 students: 50 (top city Delhi)
# 🌡️ weather : avg 24.9°C (hottest 2024-08-15)
# 🍴 zomato : 100 spots, avg-rating 3.94/5

```

Mini-2 — Progress Downloads (as_completed LIVE!)

 **Definition:** `asyncio.as_completed(tasks)` = yields the coroutines in their FINISHING order — as soon as a download completes, its progress shows right away!

 **WHY is the async version so exciting?** The threading version of `as_completed` works on futures; here you get live progress in PURE single-thread async too — just one hand! 

PYTHON

```

async def download(fid, secs):
    await asyncio.sleep(secs)
    return f"file-{fid} ({secs*1000:.0f}KB)"

async def progress_dl():
    plan = [(0, 0.3), (1, 0.1), (2, 0.2), (3, 0.15)] # (id, simulated-time)
    tasks = [download(i, s) for i, s in plan]
    t0 = time.perf_counter()
    done = 0
    for fut in asyncio.as_completed(tasks):
        result = await fut
        done += 1
        bar = "█" * done + "░" * (len(plan) - done)
        print(f" 📄 {result} aaya! progress [{bar}] {done}/{len(plan)} "
              f"({time.perf_counter()-t0:.2f}s)")

run(progress_dl())

# Expected OUTPUT (finishing-order: 0.1→0.15→0.2→0.3 – chhoti pehle!):
# 📄 file-1 (100KB) aaya! progress [██░░] 1/4 (0.10s)
# 📄 file-3 (150KB) aaya! progress [██░░] 2/4 (0.15s)
# 📄 file-2 (200KB) aaya! progress [██░░] 3/4 (0.20s)
# 📄 file-0 (300KB) aaya! progress [██░░] 4/4 (0.30s)

```

Mini-3 — TIMEOUT-SHIELD: Give the Slow API a Red-Card!

 **Definition:** `await asyncio.wait_for(coro, timeout=t)` = give the coroutine a t-second deadline — if it crosses it, you get a `TimeoutError` (cancelled!). Even if the API hangs, YOU don't — just show a fallback.

 **Mistake:** assuming a timeout "will work" without testing it against a real server — always drill with a fake-slow API first (like below!).

PYTHON

```

async def lazy_api(delay):
    await asyncio.sleep(delay)
    return "real-data-mil-gayi!"

async def shielded_call():
    # Case-A: fast API (0.1s < 0.25s timeout) → success
    try:
        res = await asyncio.wait_for(lazy_api(0.1), timeout=0.25)
        print(f"    ✅ fast-API: {res}")
    except TimeoutError:
        print(f"    (nahi aana chahiye tha yahan!)")

    # Case-B: Lazy API (0.5s > 0.25s timeout) → shield kicks in 🛡️
    try:
        res = await asyncio.wait_for(lazy_api(0.5), timeout=0.25)
        print(f"    {res}")
    except TimeoutError:
        print(f"    🛡️ SHIELD ON: lazy-API 0.25s me nahi aayi → CANCEL + fallback!")
        res = "CACHED-DATA (fallback 📄)"
        print(f"    user ko dikha: {res}")

run(shielded_call())
# Expected OUTPUT:
#    ✅ fast-API: real-data-mil-gayi!
#    🛡️ SHIELD ON: Lazy-API 0.25s me nahi aayi → CANCEL + fallback!
#    user ko dikha: CACHED-DATA (fallback 📄)

```

3 Combo-Specific Mistakes

#	❌ MISTAKE	✅ CORRECT
1	Assuming the results order (as_completed ≠ map!)	ask for the finishing order only
2	Trusting every API without wait_for	always use the shield (production!)
3	gather without try-any-Exception (everything fails)	gather(return_exceptions=True) or per-task try

🚀 Recap — 30 Seconds

- `gather` = combo-digest (time ≈ the slowest one!) 📊
- `as_completed` = live-progress (finishing order!) 📉
- `wait_for` = timeout-shield (red-card for slow APIs 🛡️)

Next: `task.ipynb` — 7 async-orders on the Order-Board + the BOSS! 🏠

01asynciobasics_theory

Level 0 — `create_task` : Take the Order and Walk Off 🏃

📖 **Definitions:**

- `gather(c1, c2)` = the waiter SITS and waits for all the orders together
- `create_task(coro)` = writes into the kitchen-ledger and LEAVES — the coroutine starts running in the background while you do the next task; `await task` whenever you need it.

🔍 **WHY?** tasks = a HANDLE to running-coroutines — you can cancel them, await them, or poll them (`task.done()`). Use `gather` for a small squad; use `create_task` for INDEPENDENT operations!

PYTHON

```
import asyncio, time
try:
    import nest_asyncio; nest_asyncio.apply()
except ImportError:
    pass

def run(coro):
    return asyncio.run(coro)

async def kitchen(dish, t):
    await asyncio.sleep(t)
    print(f"🕒 {dish} ready! ({t}s)")
    return dish

async def counter_order():
    t_dosa = asyncio.create_task(kitchen("dosa", 0.3))
    t_idli = asyncio.create_task(kitchen("idli", 0.15))
    print("🍽️ ordered both! (waiter aage badha – kaam chal raha...)")
    print("dosa done?", t_dosa.done(), "(abhi nai – chal raha!)")
    r1 = await t_dosa
    print("after await: done?", t_dosa.done(), "| got:", r1)
    r2 = await t_idli # already hone wala (0.15 < 0.3) – instant!
    print("idli:", r2, "(yeh toh pehle Hi ready tha – instant await!)")

run(counter_order())
# Expected OUTPUT:
# 🍽️ ordered both! (waiter aage badha – kaam chal raha...)
# dosa done? False (abhi nai – chal raha!)
# 🕒 idli ready! (0.15s)
# 🕒 dosa ready! (0.3s)
# after await: done? True | got: dosa
# idli: idli (yeh toh pehle Hi ready tha – instant await!)
```

Level 1 — Semaphore(2) : 5 Downloads, ONLY 2 Lanes! 🚦

📖 **Definition:** `sem = asyncio.Semaphore(n)` + `async with sem:` = at any ONE time a MAXIMUM of `n` coroutines are inside that section — the rest wait in line! 🗑️

🔍 The courtroom story: 5 files in 2 lanes → 3 batches — time ≈ slowest+middle... watch live: peak-concurrency is EXACTLY 2 (verify it with a counter!)

PYTHON

```

async def lane_demo():
    SEM = asyncio.Semaphore(2)
    live = 0
    peak = 0
    async def dl(fid, secs):
        nonlocal live, peak
        async with SEM:
            live += 1
            peak = max(peak, live)
            print(f" 📄 file-{fid} lane mil gayi (active={live})")
            await asyncio.sleep(secs)
            live -= 1
        return fid

    plan = [(0, 0.2), (1, 0.2), (2, 0.3), (3, 0.1), (4, 0.25)]
    t0 = time.perf_counter()
    got = await asyncio.gather(*(dl(i, s) for i, s in plan))
    e1 = time.perf_counter() - t0
    print(f"\n🔗 got: {got}")
    print(f" 🚦 peak concurrency = {peak} (Semaphore(2) ka vada nibhaya!)")
    print(f" ⌚ {e1:.2f}s - 2 lanes pe ~5xavg/2: na 5-parallel-0.3, na sequential-1.05!")

run(lane_demo())

# Expected OUTPUT (lane-acquire order deterministic!):
# 📄 file-0 lane mil gayi (active=1)
# 📄 file-1 lane mil gayi (active=2)
# 📄 file-2 lane mil gayi (active=1) ← 0.2s pe f0 khalaa, f2 ko line mili
# 📄 file-3 lane mil gayi (active=2) ← f1 bhi khatam, f3 lane me
# 📄 file-4 lane mil gayi (active=2) ← 0.3s pe f3 (0.1s) done → f4
# 🔗 got: [0, 1, 2, 3, 4]
# 🚦 peak concurrency = 2 (Semaphore(2) ka vada nibhaya!)
# ⌚ 0.55s (f4 0.3+0.25 = mehengi chain!)

```

Level 2 — Per-Task Timeout: Some Fail, the Rest Win! 🏆

📖 **Pattern:** news-wire — 4 "sources", each with its own `wait_for` timeout. One slow source must not kill the whole digest — `gather(return_exceptions=True)`!

PYTHON

```

async def wire(source, t, timeout=0.22):
    try:
        async def fetch():
            await asyncio.sleep(t)
            return f"📰 {source} headlines"
        return await asyncio.wait_for(fetch(), timeout=timeout)
    except TimeoutError:
        return f"🕒 {source} TIMEOUT (skip!)"

async def news_digest():
    feeds = [("Zeenews", 0.10), ("AajTak", 0.20), ("DDLazy", 0.50), ("OffersFact", 0.15)]
    print("📰 3 se zyada t waali atak jayengi (timeout=0.22)...")
    results = await asyncio.gather(*(wire(n, t) for n, t in feeds))
    for r in results:
        print(" ", r)
    ok = sum("📰" in r for r in results)
    print(f"🏆 digest: {ok}/4 sources – partial-success accepted!")

run(news_digest())
# Expected OUTPUT:
# 📰 3 se zyada t waali atak jayengi (timeout=0.22)...
# 📰 Zeenews headlines
# 📰 AajTak headlines
# 🕒 DDLazy TIMEOUT (skip!)
# 📰 OffersFact headlines
# 🏆 digest: 3/4 sources – partial-success accepted!

```

The Real World: `aiohttp` (CONCEPT — the sandbox here is offline!)

On real servers, downloads use the `aiohttp` library. Everything works exactly as you learned: 🔍 You already know every move: `async-with`, `SEM`, `gather`, `await-on-IO`. `aiohttp` is just this pattern on the real wire! (Offline-sandbox: the site won't load via `curl` here — grasp the concept, then try it on your own machine!)

PYTHON

```

import aiohttp, asyncio # pip install aiohttp

SEM = asyncio.Semaphore(3) # 3 lanes!

async def fetch(session, url):
    async with SEM: # flood-control
        async with session.get(url) as resp: # async HTTP!
            return resp.status, await resp.text() # await on network-IO

async def main():
    urls = ["https://api.github.com", "https://pypi.org/pypi/requests/json"]
    async with aiohttp.ClientSession() as s:
        reports = await asyncio.gather(*(fetch(s, u) for u in urls))
    print(reports)

asyncio.run(main())

```

5 Classic Mistakes (IO-toolkit)

#	❌ MISTAKE	✅ CORRECT
1	Create a task but forget the await (killed by GC!)	every task is <code>await</code> ed/gather'ed
2	Forget the semicolon-brother of Semaphore (<code>with</code> miss)	<code>async with SEM:</code> always
3	Using Semaphore as a global and making things slow	understand its per-lane nature
4	Timeouts set too tight (everything fails)	per-task <code>wait_for</code> granularity
5	awaiting per-task in a gather loop (serial!)	make a list → one single gather

🔴 Recap — 30 Seconds

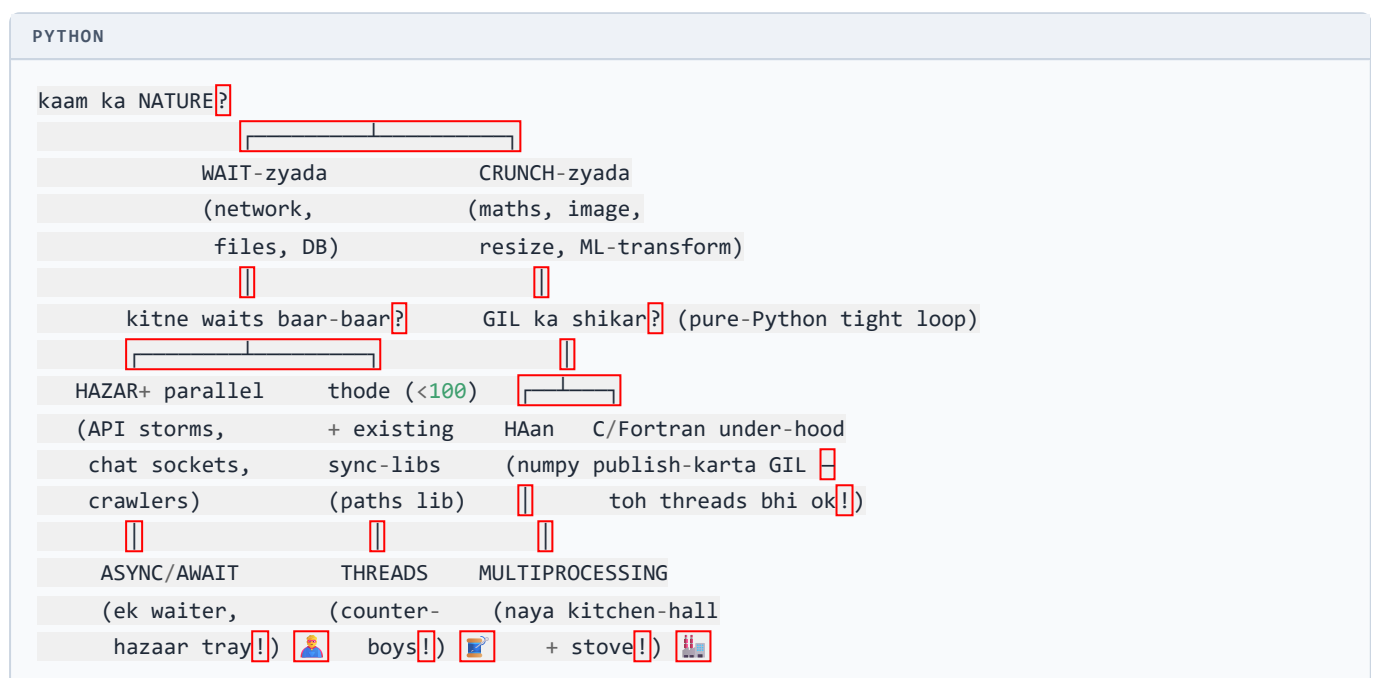
- `create_task` = take the order and move on 📄 · poll with `task.done()` · collect with `await`
- `Semaphore(n)` = n lanes 🚦 — verify deterministically with a peak-counter!
- per-task `wait_for` = partial-success digest 🏆 (one fail ≠ everything doomed)
- `aiohttp` = all of these patterns on a real server (grasp the concept offline, then try!)

01decisionguide_theory

Master Decision-Tree (not memorized — UNDERSTOOD!)

🔍 WHY does this tree work?

- During WAIT the CPU is free → 1 waiter (async) is the cheapest; 100 waiters (threads) cost more; a fresh-kitchen (process) is not even a solution for waiting!
- During CRUNCH the GIL locks things: pure-Python threads don't work; **processes-only** is the path!
- numpy/pandas release the GIL (C-level) → a favor: threads are sometimes OK!



Level 0 — 6-Scenario Quiz (think for yourself first!)

Below, guess each scenario's verdict before running it, then run the cell and see the verdict — plus WHY in two lines of reasoning!

PYTHON

```
def recommend(waits_parallel, cpu_crunch, sync_libs=False):
    """chhota decision-helper:
    waits_parallel = baar-baar wait? (bool)
    cpu_crunch     = pure-Python CPU Loop? (bool)
    sync_libs      = purani NON-async libs use kar rahe? (bool)
    """
    if cpu_crunch:
        return ("👨‍🍳 PROCESSES", "GIL-tala pure-Python – naya-kitchen-hall (mp)!")
    if waits_parallel and not sync_libs:
        return ("👨‍💻 ASYNC", "wait-storm + async-friendly libs = ek-waiter hazaar-tray!")
    if waits_parallel and sync_libs:
        return ("👨‍💻 THREADS", "sync-libs ko async nahi ban sakte – counter-boys rescue!")
    return ("👤 PLAIN", "na wait na crunch – seedha loop (khatam-kahani!)")

scenarios = [
    ("1 500 products ka REST-API scrape (httpx)", dict(waits_parallel=True, cpu_crunch=False)),
    ("2 1000 images ka custom-Python pixel-filter", dict(waits_parallel=False, cpu_crunch=True)),
    ("3 50 db-queries using OLD requests+sqlalchemy-sync", dict(waits_parallel=True, cpu_crunch=False, sync_libs=True)),
    ("4 numpy vector math 10M rows", dict(waits_parallel=False, cpu_crunch=True)), # note: C-release →
    ("5 5 audio files ka API-upload", dict(waits_parallel=True, cpu_crunch=False, sync_libs=True)),
    ("6 ek chhoti fib(30) ek-hi baar", dict(waits_parallel=False, cpu_crunch=False)),
]

for label, kw in scenarios:
    verdict, why = recommend(**kw)
    print(f"{label}\n    → {verdict} | {why}")

# Expected OUTPUT:
# 1 500 products ka REST-API scrape (httpx)
#    → 👨‍💻 ASYNC | wait-storm + async-friendly libs = ek-waiter hazaar-tray!
# ...
# 5 5 audio files ka API-upload
#    → 👨‍💻 THREADS | sync-libs ko async nahi ban sakte – counter-boys rescue!
# 6 ek chhoti fib(30) ek-hi baar
#    → 👤 PLAIN | na wait na crunch – seedha loop (khatam-kahani!)
```

Level 1 — Scorecard: All Three Tools, 6 Metrics 🏆

METRIC	👤 ASYNC	🧵 THREADS	🏗️ PROCESSES
IO-wait storms	🏆 1 🏆 1	🏆 2	🏆 3
CPU-crunch (pure-py)	❌	❌ (GIL)	🏆 1 🏆 1
scale to thousands of connections	🏆 1 🏆 1	OS-limit ~	❌ heavy
with sync-libs	❌ needs-async	🏆 1	😬
debugging ease	🏆 3 (slick)	🏆 2	🏆 1 (isolated)
code-complexity	😬 async/await everywhere	😊 easy-start	😬 IPC-hassle

PYTHON

```
# Live-mini: teeno "cost" ek-tera-feel – dispatch table!
TOOLBOX = {
    "async": {"wait_storm": 10, "cpu_crunch": 2, "hazar_conn": 10, "sync_libs": 2},
    "threads": {"wait_storm": 7, "cpu_crunch": 1, "hazar_conn": 4, "sync_libs": 9},
    "processes": {"wait_storm": 4, "cpu_crunch": 10, "hazar_conn": 1, "sync_libs": 5},
}

def score(tool, scenario_weights):
    return sum(TOOLBOX[tool][k] * w for k, w in scenario_weights.items())

challenges = {
    "API-storm": {"wait_storm": 3, "hazar_conn": 2},
    "CPU-crunch": {"cpu_crunch": 5},
    "sync-libs-io": {"wait_storm": 2, "sync_libs": 3},
}

for sc, w in challenges.items():
    board = sorted(((score(t, w), t) for t in TOOLBOX), reverse=True)
    print(f"{sc:12s} → " + " ".join(f"{t}={s}" for s, t in board),
          f" 🏆 {board[0][1].upper()}")

# Expected OUTPUT:
# API-storm → async=50 threads=29 processes=14 🏆 ASYNC (10*3+10*2!)
# CPU-crunch → processes=50 async=10 threads=5 🏆 PROCESSES
# sync-libs-io → threads=41 async=26 processes=23 🏆 THREADS
```

Level 2 — Production Combos (a mix of real apps!) 🧩

Real apps are never single-tool pure — they're a COMBO platter:

PYTHON

```
# 1. A BLOCKING-lib inside AsyncIO? Use a ThreadPool shim:
loop = asyncio.get_running_loop()
result = await loop.run_in_executor(None, blocking_func, arg)
# run sync-libs inside the async event-loop without choking!

# 2. CPU-crunch in the middle of async tasks? Use a ProcessPool executor:
result = await loop.run_in_executor(process_pool, heavy_calc, data)
# the waiter's hand stays FREE while a stove runs in the kitchen-hall!

# 3. OCR pipeline: async(download) → thread(PIL) → process(ocr-CPU)
# each stage uses its own best tool – that is production JUGAAD! 🤖
```

7 Deadly “Wrong-Tool” Mistakes

#	❌ MISTAKE	✅ CORRECT
1	CPU-crunch with threads	processes (verify-GIL-skip!)
2	thinking async with 1000-simultaneous-sync libs	threads or async-refactor (httpx!)
3	treating mp as a "speed-God" everywhere	mp is heavy: only for TRUE crunch
4	sync <code>requests.get</code> inside fastapi	<code>httpx.AsyncClient</code> or <code>run_in_executor</code>
5	<code>time.sleep</code> in async-code	<code>asyncio.sleep</code> (loop-air!)
6	lambda in a process-pool (pickle-error!)	top-level funcs (<code>if __name__ == ...</code>)
7	skipping the scenario-quiz and memorizing	build your own guide (print this!)

🔴 Recap — Section-12's Golden Frame

- Big WAIT-Storm → **ASYNC** 🤖 (one waiter, Semaphore + wait_for shields!)
- Few IO sync-libs → **THREADS** 🏪 (counter-boys, the GIL doesn't interfere!)
- A CRUNCH-life → **PROCESSES** 🏢 (a new hall, `__main__` guard!)
- combos = the production-platter 🍽️ · PEP/GIL keeps SHIFTING all your life (this guide stays alive!)

Section 12 = COMPLETE! 🎓 Next section: OOPS's five doors (13OOPSConcepts) 🚪

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- CPU-bound vs I/O-bound reasoning
- threads and the GIL
- multiprocessing and process pools
- asyncio, await and event loops

- choosing the right concurrency model

EXAM AUR INTERVIEW PERSPECTIVE

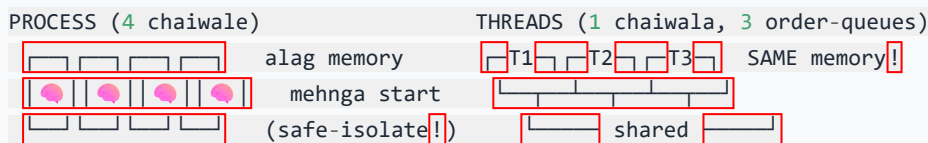
Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

LEVEL 0 — PROCESS VS THREAD: THE DNA DIFFERENCE 🧬



PYTHON

LEVEL 0 — YOUR FIRST COROUTINE 🍵

```
order = serve_chai()      # ❌ you only got the bundle – no tea was made!
order = await serve_chai() # ✅ the waiter actually made the tea
```

PYTHON

LEVEL 1 — 'ASYNCIO.GATHER': ALL ORDERS TO THE KITCHEN IN ONE GO! 🍳

```
sync: [chai-1 0.5s] → [chai-2 0.5s] → [chai-3 0.5s] = 1.5s
gather: [chai-1 ch]
       [chai-2 ch] | sab-overlap = 0.5s ⚡ (3x!)
       [chai-3 ch]
```

PYTHON

LEVEL 2 — PRODUCTION COMBOS (A MIX OF REAL APPS!) 🍷

```
# 🔗 1. A BLOCKING-lib inside AsyncIO? Use a ThreadPool shim:
loop = asyncio.get_running_loop()
result = await loop.run_in_executor(None, blocking_func, arg)
# run sync-libs inside the async event-loop without choking!

# 🔗 2. CPU-crunch in the middle of async tasks? Use a ProcessPool executor:
result = await loop.run_in_executor(process_pool, heavy_calc, data)
# the waiter's hand stays FREE while a stove runs in the kitchen-hall!

# 🔗 3. OCR pipeline: async(download) → thread(PIL) → process(ocr-CPU)
# each stage uses its own best tool – that is production JUGAAD! 🤖
```

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

☐ Level 0 — Process vs Thread: The DNA Difference 🦋

☐ Level 1 — IO-BOUND vs CPU-BOUND: Which Task, Which Kind of Exhaustion? 🤖

☐ Level 2 — GIL: Python's Single-Key Lock 🔑

☐ Level 3 — Multiprocessing: Bring in 4 Tea-Stall Owners! 🍵

☐ 5 Classic Mistakes (concurrency edition)

☐ Race-1 — 10 Downloads, Two Teams

☐ map vs as_completed — Two Ways of a Postman

☐ Race-2 — GIL-Plateau LIVE: 1 vs 2 vs 4 CPU-Threads

☐ Final Scorecard + When-to-Use-What Cheat-Matrix

☐ Recap — 30 Seconds

☐ Level 0 — Your First Coroutine 🍷

☐ Level 1 — `asyncio.gather`: ALL Orders to the Kitchen in One Go! 🍷

☐ Level 2 — The Storm of 5 Fake-API Calls 🌩️ (working with results!)

☐ 5 Classic Mistakes (async edition)

☐ Setup + universal run() (the key from the previous notebook!)

☐ Mini-1 — API-DASH: 3 APIs, One Digest

☐ Mini-2 — Progress Downloads (as_completed LIVE!)

☐ Mini-3 — TIMEOUT-SHIELD: Give the Slow API a Red-Card!

☐ 3 Combo-Specific Mistakes

☐ Level 0 — `create_task`: Take the Order and Walk Off 📄

☐ Level 1 — `Semaphore(2)`: 5 Downloads, ONLY 2 Lanes! 🚦

☐ Level 2 — Per-Task Timeout: Some Fail, the Rest Win! 🏆